

Regular Expressions in Programming Languages: A Peek at Python



This new series of articles will take readers on a journey that explores the regular expressions in various programming languages. The first article in this series takes a detailed look at the use of regular expressions in Python.

Often when people discuss regular expressions, they use the term ‘regex’, which leads to a mix up between regular expressions and the tools that use regular expressions for string searching. A classic example of this sort of mistake is people confusing regular expressions and *grep*, a UNIX utility for pattern matching. *grep* is a very powerful tool but whatever it may be, it definitely is not a synonym for regular expressions. A regular expression is not a tool but rather, a concept in formal language theory that defines how a sequence of characters can define a search pattern. On the other hand, *grep* is a tool that uses regular expressions for pattern matching. But there are a lot of other utilities and programming languages that use regular expressions for pattern matching.

The main problem with regular expressions is that the syntax as well as the manner of calling regular expressions differs for the various languages, with some of them closely resembling each other, while others having major differences between them. So, in this series of articles, we discuss how to use regular expressions in the following six programming languages — Python, Perl, Java, JavaScript, C++ and PHP. This doesn’t mean that these are the only programming languages or software that support regular expressions. There are a lot of others that do. For example, you can find regular expressions in programming languages like AWK, Ruby, Tcl, etc, and in software like sed, MySQL, PostgreSQL,

etc. Moreover, even absolute beginners typing **.pdf* in their search boxes on Windows to search all the PDF files in their system are using regular expressions. Since a lot of articles about regular expressions cover *grep* in detail, this article will not cover the most famous tool that uses regular expressions. In fact, the one tool that solely depends on regular expressions for its survival is *grep*. The main reason one needs to study regular expressions is that many results obtained from powerful data mining tools like Hadoop and Weka can often be replicated by using simple regular expressions.

Some of the popular regular expression syntaxes include Perl-style, POSIX-style, Emacs-style, etc. The syntax of the regular expression used in a tool or programming language depends on the regular expression engine used in it. The ability to use more than one regular expression engine in a single tool itself leads to the support of more than one regular expression style. For example, the immensely popular regular expression tool by GNU called *grep*, by default, uses a regular expression engine that supports POSIX regular expressions. But it is also possible to use Perl-style regular expressions in *grep* by enabling the option *-P*. In Perl-style regular expressions, the notation *\d* defines a pattern with a digit, whereas in POSIX-style regular expressions, this regular expression does not have any special meaning. So, in the default mode of this regular expression, it will match the letter